

ERA Tutorium 9

Leopold Cario	<u>Termin:</u>	<u>Raum:</u>	<u>Zulip:</u>
	· Mittwoch 10:00	03.13.010	ERA Tutorium - Mi - 1000-1
	· Donnerstag 12:00	00.08.059	ERA Tutorium - Do - 1200-1

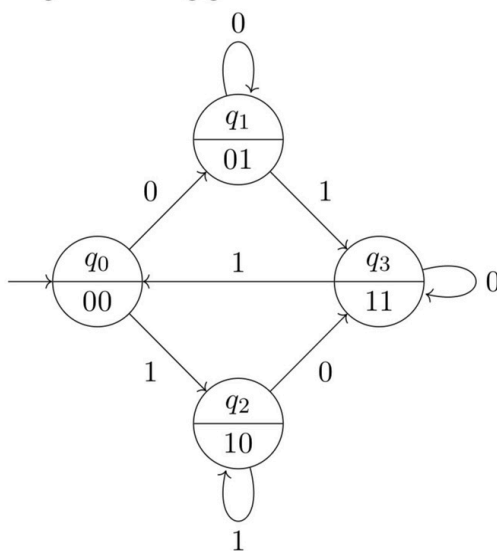
Vorlesungsevaluation : Tutorium Gruppe 9

- Repräsentiert Funktion einer sequentiellen Schaltung (sequentiell: zustandsabhängig)
- als Diagramm: Zustände $\rightarrow$ Kreise, Übergänge $\rightarrow$ Kanten, Bedingungen $\rightarrow$ Kantenbeschriftungen
- als 6-Tupel $(I, O, S, s_0, \delta, \lambda)$:
 - I : Menge möglicher Eingaben
 - O : Menge möglicher Ausgaben
 - S : Zustandsmenge
 - s_0 : Startzustand
 - $\delta : S \times I \rightarrow S$: Zustandsübergangsfunktion
 - $\lambda : S \rightarrow O$ (Moore), $\lambda : S \times I \rightarrow O$ (Mealy): Ausgabefunktion

Endliche Automaten: Beispiele

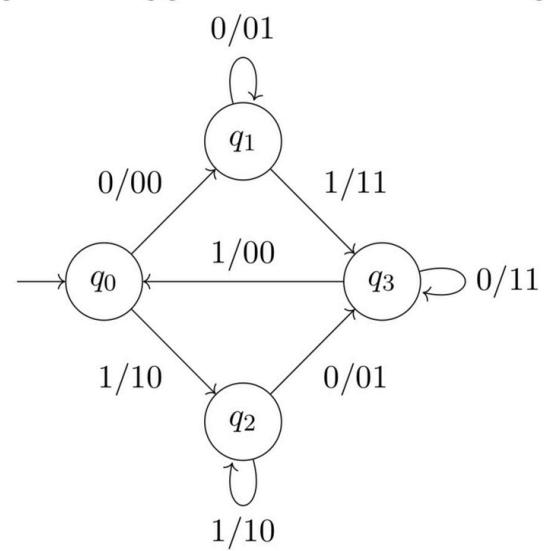
Moore-Automat

Ausgabe abhängig von aktuellem Zustand



Mealy-Automat

Ausgabe abhängig von aktuellem Zustand + Eingabe



$I = \{0, 1\}, O = \{00, 01, 10, 11\}, S = \{q_0, q_1, q_2, q_3\}, \delta, \lambda$ (abh. vom Typen)

Endliche Automaten: Realisierung

- One-Hot-Kodierung: Genau 1 FF ist auf 1 (aktueller Zustand), einfach aber verschwenderisch
- Binärkodierung: FFs zusammen bilden Binärzahl des aktuellen Zustands, spart FFs aber komplexer
- Mikroprogrammierte Steuerwerke: Nur ein Speicherbaustein, enthält vollständigen Automaten. Eingaben werden als Adressen interpretiert, sehr flexibel.

Zustand	One-Hot	Binär
S_0	0001	00
S_1	0010	01
S_2	0100	10
S_3	1000	11

Lehrstuhl für
Rechnerarchitektur & Parallele Systeme
Prof. Dr. Martin Schulz
Dominic Prinz
Jakob Schäffeler

Lehrstuhl für
Design Automation
Prof. Dr.-Ing. Robert Wille
Stefan Engels
Benjamin Hien

Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

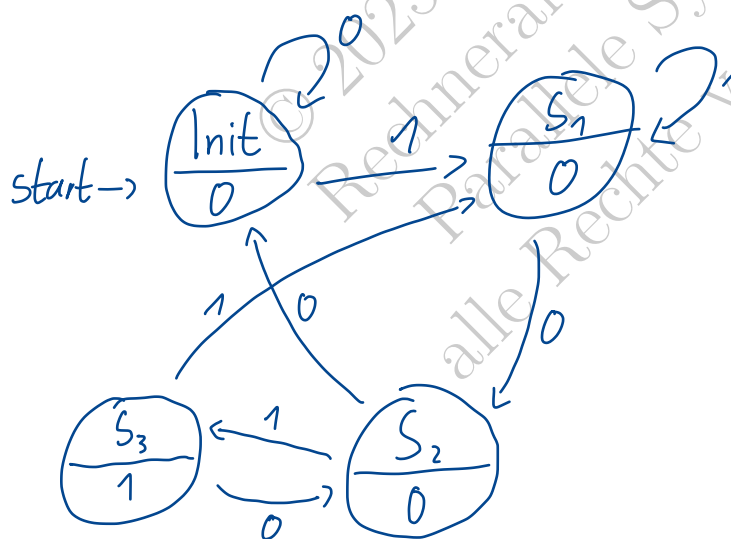
Übungsblatt 9: RISC-V Multicycle Prozessor

15.12.2025 – 19.12.2025

1 Pattern Recognizer

Entwerf eine Schaltung, welche die Bitfolge 101 in einem Bitstrom I erkennt. Die Schaltung sollte dazu immer $O = 1$ ausgeben, wenn die Bitfolge aufgetreten ist. Ansonsten sollte $O = 0$ ausgegeben werden. Führe dazu folgende Schritte durch:

- a) Entwerf das Zustandsdiagramm als Moore-Automat.



01110001101010

- b) Beschreibe den endlichen Automaten formal als 6-Tupel $A(I, O, S, s_0, \delta, \lambda)$, wobei I die Menge der Eingänge, O die Menge der Ausgänge, S die Menge der Zustände, s_0 den Anfangszustand, δ die Übergangsfunktion und λ die Ausgabefunktion bezeichnet.

$$\cdot I = \{0, 1\}$$

$$\cdot O = \{0, 1\}$$

$$\cdot S = \{Init, S_1, S_2, S_3\}$$

$$\cdot s_0 = Init$$

$$\cdot \delta =$$

$$\cdot \lambda =$$

- $\delta : S \times I \rightarrow S$ definiert durch

- $\delta(Init, 0) = Init$

- $\delta(Init, 1) = S_1$

- $\delta(S_1, 0) = S_2$

- $\delta(S_1, 1) = S_1$

- $\delta(S_2, 0) = Init$

- $\delta(S_2, 1) = S_3$

- $\delta(S_3, 0) = S_2$

- $\delta(S_3, 1) = S_1$

- $\lambda : S \rightarrow O$ definiert durch:

- $\lambda(Init) = 0$

- $\lambda(S_1) = 0$

- $\lambda(S_2) = 0$

- $\lambda(S_3) = 1$

- c) Überführe das Zustandsdiagramm in eine Wahrheitstabelle und anschließend in Boolesche Terme. Nutze hierzu die Binärdkodierung.

$\cdot \text{Init} = 00$
 $\cdot S_1 = 01$
 $\cdot S_2 = 10$
 $\cdot S_3 = 11$

I	FF1	FF0	FF1'	FF0'	O
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	0	0	0
0	1	1	1	0	1
1	0	0	0	1	0
1	0	1	0	1	0
1	1	0	1	1	0
1	1	1	0	1	1

$$FF1' = \bar{I} \cdot \overline{FF1} \cdot FF0 + \bar{I} \cdot FF1 \cdot \overline{FF0} + I \cdot FF1 \cdot \overline{FF0}$$

$$= \underline{\bar{I} \cdot FF1 \cdot \overline{FF0} + \bar{I} \cdot FF0}$$

$$FF0' = \underline{I}$$

$$O = \underline{FF1 \cdot FF0}$$

- d) Entwirf die entsprechende Schaltung in Digital, einmal Synthese der konkreten Funktionalität und einmal als mikroprogrammiertes Steuerwerk. Nutze hierzu die Binärdkodierung.

© 2025 Lehrstuhl für
Rechnerarchitektur &
Parallele Systeme
alle Rechte vorbehalten

2 Prozessor Performance

Nach Anlegen eines Inputs braucht es eine gewisse Zeit bis eine Veränderung an den Outputs eines digitalen Bauteils sichtbar ist. Der Delay ist die Zeit die ein Input braucht um verarbeitet zu werden. Folgende Tabelle zeigt Delays einzelner Bauteile die im RISC-V Prozessor verwendet werden. Delays für Bauteile die hier nicht angegeben sind, dürfen als 0 angenommen werden.

Element	Parameter	Delay (ps)
Register read	t_{RegRead}	40
Register setup	t_{RegSetup}	50
Multiplexer	t_{mux}	30
AND-OR gate	$t_{\text{AND-OR}}$	20
ALU	t_{ALU}	120
Decoder (Control Unit)	t_{dec}	25
Extend unit	t_{ext}	35
Memory read	t_{MemRead}	200
Memory setup	t_{MemSetup}	150
Register file read	t_{RFRead}	100
Register file setup	t_{RFSetup}	60

Tabelle 1: Propagation Delays

Hinweis: t_{dec} berücksichtigt nur die Zeit, um die Kontrollsignale aus dem Input zu dekodieren. Beachte, dass beim Multi-Cycle Prozessor die Kontrollsignale eindeutig aus dem State hervorgehen. Allerdings ist ein Register read notwendig, damit der aktuelle State anliegt. Die effektive Zeit der Multi-Cycle Control Unit ist daher $t_{\text{RegRead}} + t_{\text{dec}}$.

a) Bestimme die minimale Taktlänge $T_{\text{c_single}}$ des Single-Cycle RISC-V Prozessors.

b) Bestimme die minimale Taktlänge $T_{\text{c_multi}}$ des Multicycle RISC-V Prozessors.

c) Angenommen der Prozessor führt ein Programm aus, das aus 25% `lw`, 10% `sw`, 11% `beq`, 2% `jal` und 52 % R- oder I-Typ ALU Instruktionen besteht¹. Was ist die vorraussichtliche Laufzeit wenn das Programm aus 10^{11} Instruktionen besteht. Berechne das Ergebnis sowohl für den Single-Cycle RISC-V als auch den Multicycle RISC-V.

¹Dies ist ungefähr die Verteilung, mit der die genannten Befehlstypen im SPECINT2000 Benchmark vorkommen

o lw: 5
sw: 4
beq: 3
jal: 4
R-7-1yp ALU: 4

$$0,25 \cdot 5 + 0,1 \cdot 4 + 0,11 \cdot 3 + 0,02 \cdot 4 + 0,52 \cdot 4 \\ = \underline{4,14} \text{ Zyklen}$$

$$4,14 \cdot 10^{11} \cdot 375 \text{ps} = \underline{\underline{155 \text{s}}}$$

o

$$10^{11} \cdot 750 \text{ps} = \underline{\underline{75 \text{s}}}$$

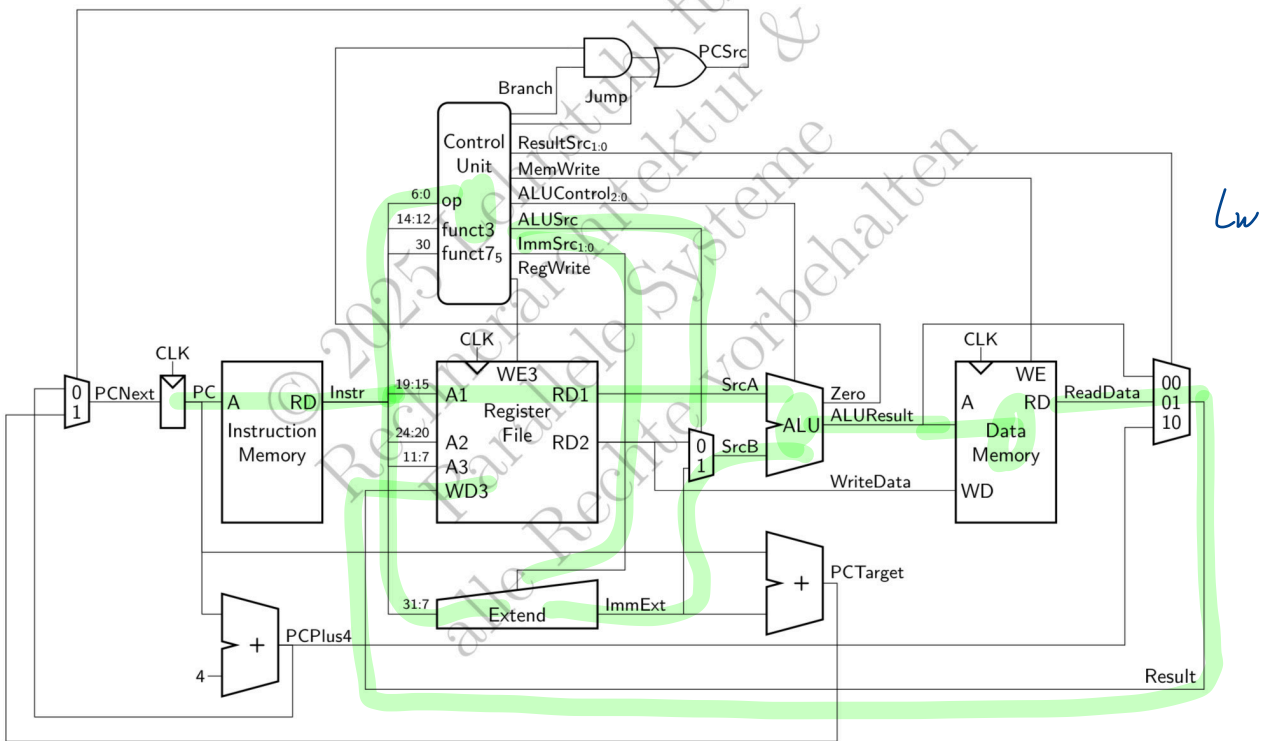


Abbildung 4: Schaltbild des Single-Cycle RISC-V Prozessors

$$T_{c-single} = t_{RegRead} + t_{mem} + t_{RegRead} + t_{ALU} + t_{mem} + t_{mux} + t_{RegSetup} = \underline{\underline{750ps}}$$

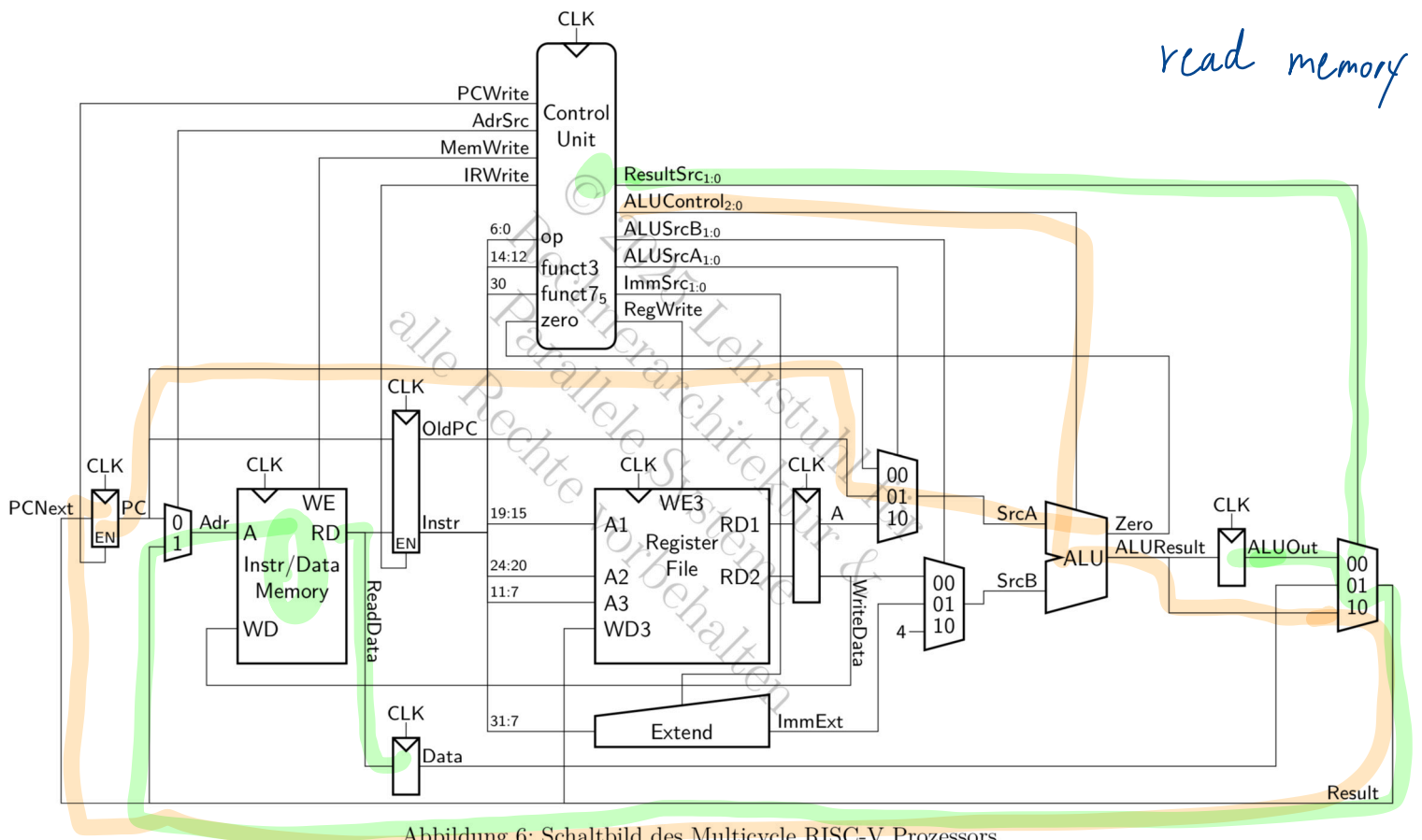


Abbildung 6: Schaltbild des Multicycle RISC-V Prozessors

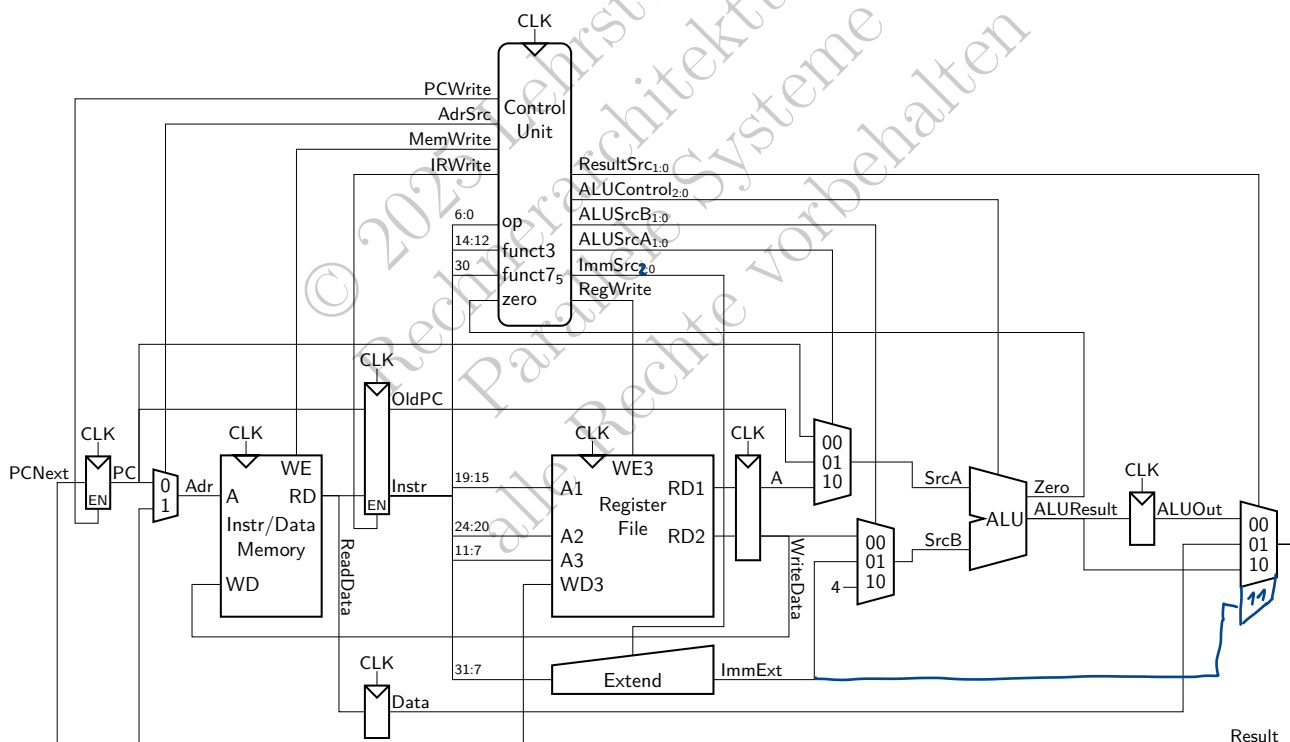
$$T_{c-multi} = t_{RegRead} + t_{dec} + 2t_{mux} + t_{mem} + t_{RegSetup} = \underline{\underline{375ps}}$$

- d) Das Delay eines der verwendeten Komponenten soll verringert werden um die Taktrate des Multicycle RISC-V Prozessors zu beschleunigen. Welches Bauteil sollte neu entworfen werden um die größte Verbesserung zu erhalten? Wie schnell (Delay in ps) sollte das Bauteil dann mindestens sein, also ab welchem Delay bewirken Verbesserungen nichts mehr? Wie ändert sich die Laufzeit des Programms aus der vorigen Aufgabe mit den Anpassungen?

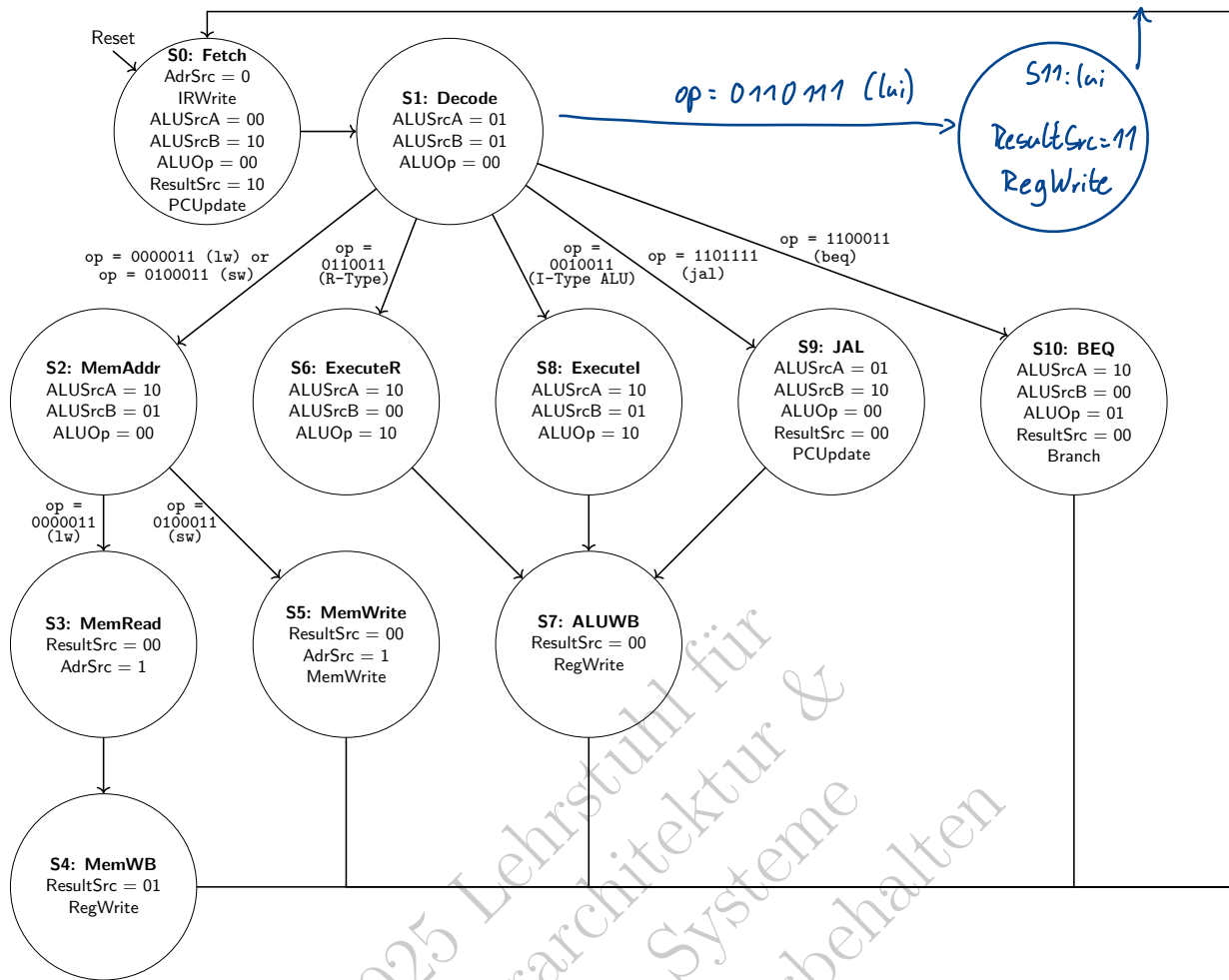
3 Prozessorerweiterung

Der Prozessor in Abbildung 6 soll erweitert werden sodass er auch den Befehl `lui` ausführen kann.

- a) Erweitern Sie den Prozessor, sodass er `lui` durchführen kann. Falls einzelne Komponenten ebenfalls angepasst werden müssen, so reicht es aus in Worten oder als Tabelle anzugeben, wie sich das Verhalten ändert.



- b) Passe den Automaten des sequentiellen Steuerwerks in **Abbildung 5** ebenfalls an. Falls neue Zustände und Signale eingeführt oder verändert werden, beachte, dass zusätzliche Signale eventuell in anderen Zuständen ebenso berücksichtigt werden müssen.



4 Patentstreit (Hausaufgabe)

Bearbeitung und Abgabe der Hausaufgabe 9 auf <https://artemis.tum.de/courses/516> bis Sonntag, den 21.12.2025, 23:59 Uhr.



Abbildung 1: Weihnachtsmann & Co. KG ©RTL Television GmbH

Der Elf Jordi von der *Weihnachtsmann & Co. KG* (kurz WM & Co. KG) möchte einen multicycle RISC-V Prozessor mit einer integrierten 16-Segment-Anzeige entwerfen. Auf der 16-Segment-Anzeige sollen Buchstabe für Buchstabe vorprogrammierte Texte erscheinen. Es wurde beschlossen, dass S-Typ Befehle, die in Adresse 0 speichern würden, zur 16-Segment-Anzeige umzuleiten. In diesem Fall wird die Anzeige entsprechend der letzten 5 bit gesteuert